

BOOP: Write *Right* Code

Vaani Goenka and Aalok Thakkar

Ashoka University

Abstract. Novice programmers frequently adopt a syntax-specific and test-case-driven approach, writing code first and adjusting until programs compile and test cases pass, rather than developing correct solutions through systematic reasoning. AI coding tools exacerbate this challenge by providing syntactically correct but conceptually flawed solutions. In this paper, we address the question of developing correctness-first methodologies to enhance computational thinking in introductory programming education. To this end, we introduce BOOP (Blueprint, Operations, OCaml, Proof), a structured framework requiring four mandatory phases: formal specification, language-agnostic algorithm development, implementation, and correctness proof. This shifts focus from making the code *work* to understanding *why* the code is correct.

BOOP was implemented at Ashoka University using a VS Code extension and preprocessor that enforces constraints. Initial evaluation shows improved algorithmic reasoning and reduced trial-and-error debugging. Students reported better edge case understanding and problem decomposition, though some initially found the format verbose. Instructors observed stronger foundational skills compared to traditional approaches, suggesting that structured correctness-first approaches may significantly improve students' computational thinking abilities.

Keywords: Correct-by-Construction Programming · Formal Methods · Program Design · Programming Proofs · Programming Tools

1 Introduction

Computer Science, at its core, is a discipline of abstraction, logic, and problem-solving. Writing code is a powerful tool to express these ideas. Yet, we frequently note that students beginning their journey in computer science at Ashoka University conflate the tool with the discipline and adopt a predominantly program-centric mindset. This manifests in several ways: students incorrectly confuse the expected behaviour of the program on a small set of test cases with total functional correctness, attempt to mechanically patch incorrect programs instead of reasoning from first principles, and develop a syntax-specific mindset in programming. In general, students even tend to overestimate their understanding of program constructions [25]. This is reflected in their prioritisation of *coding a solution* over understanding the underlying algorithm. Roundabout approaches often result in students only being able to consider test cases that are over-fitted to their code, disregarding the problem's specifications and overall correctness.

The ACM 2023 guidelines for undergraduate computer science education explicitly recommend instruction in proof-writing and correctness verification, including loop invariant application for algorithm correctness and inductive methods for recursive algorithm analysis [24].

This is more prominent in the age of AI-based programming tools and copilots. Students unfamiliar with the computational thinking process are tempted to consult language models for answers to programming problems [34]. However, language models churn out poor-quality code, and sometimes even inaccurate statements, which leads to poor programming practices [32, 3]. This worsens the student’s skill in and outlook on programming. They develop misconceptions about the necessity of learning problem-solving skills when tools such as Large Language Models (LLMs) could generate solutions for them, trusting these tools much more than they call for [32].

It is imperative to shift the focus in introductory CS courses from ‘writing a program’ to ‘writing *right* programs.’ This paper explores how instructors can adopt a novel computer science pedagogy paradigm that promotes accurate best-practice programming by design.

2 Related Work

Computing education research has extensively documented the challenges students face when learning introductory programming. Students step in with pre-conceptions about programming difficulty or develop negative attitudes early in their learning journey, leading to ineffective learning approaches such as rote memorization and, in some cases, academic dishonesty.

A common pedagogical intervention is reducing extraneous cognitive load while preserving the essential problem-solving aspects of programming instruction. Constrained environments like mini-languages (turtle graphics) and block-based systems (MIT’s LogoBlocks [4]) mimic basic programming concepts like achieving a goal by breaking it down into a correct sequence of instructions. These approaches use syntactic constraints as scaffolding and support “hybrid programming environments that blend block-based and text-based tools” [37].

However, fundamental programming concepts must not be over-abstracted to the point that application and formalization become disconnected. While referential frameworks can provide an initial conceptual understanding, students’ learning and evaluation should be guided by a more robust and well-structured system that comprises both formal and informal methods.

2.1 Functional Programming Education

Research identifies the most challenging aspects of programming for students as: (1) debugging programs, (2) designing programs to solve specific problems, and (3) decomposing functionality into procedures, all of which are fundamental to functional programming [25, 15]. Additionally, students reported that 2 of their

top 3 difficulties were error handling and recursion. Moreover, instructors highlighted the need for more problem-solving-oriented pedagogical materials [25, 16]. On the topic of problem-solving, Hudak, a pioneer of the International Haskell Committee, emphasizes the importance of developing problem specifications before implementation details while acknowledging the need for effective compilation techniques to make functional languages practical for education [19]. These are key ideas that guide our research.

2.2 AI and CS Pedagogy

Recent research has examined the intersection of artificial intelligence tools and programming education, revealing significant pedagogical challenges. While LLMs perform well with high-resource languages like Python, their capabilities with low-resource languages such as OCaml remain limited [6]. Even when ChatGPT provides correct solutions to functional programming problems in OCaml 68% of the time, only half of these solutions are pedagogically valuable or “benefit students in some form”[32]. Moreover, ChatGPT-generated code exhibits detectable patterns that even remain consistent across prompt variations[6]. Accordingly, researchers recommend emphasizing code comprehension, debugging, and correctness within structured design frameworks [34, 10].

2.3 Pattern-Based Approaches to Programming Education

Educational research has also explored pattern-based pedagogical frameworks such as POSA and GoF [5, 11]. These patterns encapsulate successful solution strategies by describing problem contexts, competing forces, and balancing structures. Although these ideas are valuable, they take away from the autonomy and worthwhile struggle of a student. A pedagogical approach must emphasize independent pattern discovery and understanding over pattern memorization, prioritizing the development of transferable algorithmic thinking skills.

2.4 Correctness-by-Construction

Several frameworks for stable, correct-by-construction programming have emerged. Meyer’s Design By Contract is a systematic approach to designing programs where preconditions, postconditions, and invariants are explicitly defined for each component [29]. Wing explicitly mentions that computational thinking involves using “invariants to describe a system’s behaviour succinctly and declaratively” [38]. Gries and Schneider emphasize the use of a rigorous, equational style of proving and show how logic can be applied to formally develop programs from specifications [14, 13]. Loksa et al. investigated meta-cognitive awareness in novice programmers by identifying six problem-solving stages and explicitly coaching students to recognize which stage they occupied when encountering difficulties [28]. A study which leveraged their framework to investigate common errors found that successful students differed primarily in forming correct initial

problem conceptualization, corresponding to the first stage: *reinterpret the problem prompt* [33]. There remains a need to integrate formal specification methods with novice-friendly structured learning environments for CS education.

3 Overview

In this section, we demonstrate our framework through a representative programming task from the Introduction to Computer Science course at our institution. The selected exercise exemplifies the challenges inherent in introducing formal specification-driven programming to CS students:

The natural numbers can be defined as: `type nat = Zero | Succ of nat`. Define a function `div` such that it takes two natural numbers `a` and `b`, and returns the quotient and the remainder when `a` is divided by `b`.

This problem operates within a constructive type system based on Peano arithmetic, where natural numbers are represented inductively. The type `nat` compels students to explicitly engage with recursive decomposition and constructive reasoning that is often implicit in conventional arithmetic operations. The given problem statement is deliberately made to omit several details: it provides no guidance on handling edge cases such as division by zero (whether through error handling, option types, or alternative approaches), does not specify the explicit type signature for the `div` function, and does not offer concrete test cases. This ambiguity forces students to engage in specification refinement as a prerequisite to implementation, requiring them to formalise their understanding of the problem domain before attempting a solution.

3.1 Three Incorrect Approaches

We first analyse student submissions for this question. Three representative submissions are presented in Appendix A. We observe the following misconceptions:

1. **Imperative State Tracking:** Students rightly but inadequately conceptualize division as iterative subtraction, tracking only the quotient, failing to account for the remainder.
2. **Incomplete Edge Case Handling:** Implementations exhibit unhandled division by zero due to conflation of validity of statements with their mathematical contexts.
3. **Lack of Constructive Foundation:** Students incorrectly invoke `int` operators for `nat` types, demonstrating incomplete understanding of custom types, type safety, and program requirements.

3.2 Pedagogical Analysis

```

1 type nat = Zero | Succ of nat
2
3 let plus (a: nat) (b: nat) : nat =
4   match a with
5   | Zero -> b
6   | Succ a' -> Succ (plus a' b)
7
8 let mult (a: nat) (b: nat) : nat =
9   match b with
10  | Zero -> Zero
11  | Succ b' -> plus (mult a b') a
12
13 let less_than (a: nat) (b: nat) : bool =
14   match (a, b) with
15   | (_, Zero) -> false
16   | (Zero, Succ _) -> true
17   | (Succ a', Succ b') -> less_than a' b'

```

Listing 1: Definition of addition, multiplication, and comparison on custom-defined natural numbers in OCaml.

These errors reveal critical junctures in the learning process. The intuitive notion of addition as repeated successors or multiplication as repeated addition translates seamlessly into programming (as shown in Listing 1). That is, students can conceptualise `mult` as iteratively adding `a` to itself `b` times, and the implementation follows naturally from this mental model.

Division, however, is not merely repeated subtraction. It requires simultaneously tracking how many times the subtraction occurred (the quotient). This dual-tracking requirement fundamentally challenges students' sequential thinking patterns. Moreover, students frequently carry over mathematical intuitions that do not translate directly into program logic. The type `nat` is bounded below by `Zero`, requiring all arithmetic operations to explicitly handle edge cases. Unlike multiplication (which is safe within the `nat` domain), subtraction and division require explicit safety checks and totality considerations. Furthermore, students tend to underspecify edge cases: while division by zero is rarely encountered in ordinary mathematical contexts, programming demands exhaustive case analysis. This transition from partial mathematical reasoning to total computational specification constitutes a fundamental epistemological shift.

3.3 Blueprint Specification

Addressing this shift requires explicit mathematical specification as a pedagogical intervention. Leslie Lamport compares the lack of specifications to building “a skyscraper without a blueprint” [26]. And yet, programmers routinely construct complex systems without formal specifications [26, 35, 30, 31]. Research in CS ed-

ucation consistently shows that students who engage with formal preconditions and postconditions develop robust problem-solving skills [29, 38, 14].

By requiring students to articulate precise mathematical relationships before implementation, we address what Soloway and Spohrer (1989) identify as the *specification problem*: the tendency for novice programmers to conflate problem understanding with solution construction [36].

For our running example, we specify the *blueprint*, that is, the correctness criteria, as a function contract. Function contracts, rooted in Hoare’s axiomatic semantics [17], express computational behaviour through requires clauses or preconditions (what must be true before execution) and ensures clauses or postconditions (what will be true after execution), forming a logical triple $\{P\}S\{Q\}$ where P is the precondition, S is the statement, and Q is the postcondition. This formalism compels students to reason explicitly about program correctness before engaging with the implementation details.

To write a correct `div` function, we specify the following:

1. **Requires:** `a : nat`, `b : nat`, and `b <> Zero` (that is `b` is non-zero).
2. **Ensures:** Returns a pair `(q, r)` such that:
 - (a) `a = plus (mult b q) r`
 - (b) `less_than r b = true`

For clarity and convenience, we will overload familiar arithmetic operators such as `+`, `*`, and `<` with the `nat` operations as presented in Listing 1.

3.4 Algorithm as Operations

The algorithmic insight follows Gries’ methodology in *The Science of Programming* [13]: treat the first ensures clause `a = (b * q) + r` as an invariant that must be maintained throughout the computation, while systematically working toward satisfying the second clause `r < b` through repeated subtraction. This transforms the abstract division specification into concrete operation steps:

1. With `q = Zero` and `r = a`, the first ensures clause (2a) holds. In case if `a < b`, then both the ensures clauses hold and the function can return `q = Zero` and `r = a`.
2. If `a < b`, then since division is fundamentally repeated subtraction, we *subtract* `b` from `a` to get `a'` such that `a' + b = a`. We need to define a subtraction operator that can perform this step.
3. Now suppose our function `div` can correctly divide `a'` by `b` and obtain a quotient `q'` and remainder `r'` such that
 - (a) `a' = (b * q') + r'`
 - (b) `r' < b`.

Assuming the correctness of `plus` and `mult` operators, we construct the solution to the original problem: since `a = a' + b` and `a' = (b * q') + r'`, we have:

```

1 let rec minus (n: nat) (m: nat) : nat =
2   match n, m with
3   | _, Zero -> n
4   | Zero, _ -> failwith "minus: function requires n >= m"
5   | Succ n', Succ m' -> minus n' m'
6
7 let rec div (a: nat) (b: nat) : (nat * nat) =
8   match b with
9   | Zero -> failwith "div: division by zero"
10  | _ -> safe_div a b
11
12 and safe_div (a: nat) (b: nat) : (nat * nat) =
13   match (less_than a b) with
14   | true -> (Zero, a)
15   | false ->
16     let (q', r') = safe_div (minus a b) b in (Succ q', r')

```

Listing 2: A translation of the operational steps for division and safe subtraction on natural numbers in OCaml syntax.

$$\begin{aligned}
 a &= ((b * q') + r') + b \\
 &= (b * q') + (r' + b) \\
 &= (b * (\text{Succ } q')) + r'
 \end{aligned}$$

Therefore, the quotient for `div a b` is `Succ q'` and the remainder is `r'`.

These operation steps capture the crux of algorithmic thinking in computer science: the translation of abstract specifications into precise, language-agnostic procedures. They demonstrate how to *systematically derive a correct implementation from mathematical reasoning*. The operational steps can be translated faithfully into any programming paradigm, whether functional, imperative, or object-oriented, because they encode the fundamental logical structure of the division algorithm. In Listing 2 we present it in OCaml.

This solution handles all cases appropriately, maintains readability, and lends itself to formal verification. One can easily adapt these operations for imperative implementations using loops and mutable variables if functional programming is not the target paradigm (as shown in Appendix C). Moreover, this algorithmic structure generalizes beyond natural numbers to any Euclidean domain where division with remainder is well-defined. The core operations of comparison, subtraction, and iterative reduction remain conceptually identical.

We note that such an approach may not always be the most efficient in terms of computational complexity. In our pedagogical framework, we deliberately prioritize correctness over efficiency. This emphasis aligns with established principles in CS education: Knuth argued that premature optimisation is the root of all evil [21] and emphasized that programs should be written primarily for human comprehension, with efficiency as a secondary consideration [20]. Research by Soloway et al. demonstrates that novice programmers who focus on

correctness first develop stronger debugging skills and produce robust and more maintainable code [36].

3.5 Proof of Correctness

We have designed the code to be correct by construction, following the mathematical specification directly through recursive decomposition. While testing can be used to identify errors, and indeed, *to show the presence of bugs*, it can never demonstrate their absence. In contrast, a formal proof establishes that the specification holds for all valid inputs, regardless of how many test cases are considered. In this case, the correctness of the function `div` is not merely a design intuition or observed behaviour but a theorem to be proven:

Theorem 1. *The function `div` in Listing 2 correct; that is, for given natural numbers $a : \text{nat}$ and $b : \text{nat}$:*

1. *If $b = \text{Zero}$, then the function raises a runtime error (division by zero).*
2. *If $b \neq \text{Zero}$, then `div a b` returns a pair (q, r) such that:*
 - (a) $a = (b * q) + r$, and
 - (b) $r < b$.

This theorem asserts that the program satisfies its specification (blueprint) in all well-defined cases. To establish this, we proceed by structural induction, analysing the recursive structure of the function and verifying that both post-conditions are preserved at each step. A complete proof, as expected of students in our Introduction to Computer Science course, is provided in Appendix B.

This methodology exemplifies our approach to teaching systematic program development. We want students to learn this disciplined thinking process:

1. Establish a precise **blueprint** through a function contract that has a *requires* and an *ensures* clause (or a loop contract with invariants).
2. Develop language-agnostic **operational steps** that describe the algorithm.
3. Translate these steps into program syntax (in our case, in **OCaml**) while preserving the logical structure.
4. Provide a **proof** that guarantees correctness.

This sequence embodies the principle of correct-by-construction programming, where careful specification and methodical decomposition lead naturally to verifiable implementations. By internalizing this systematic approach, students develop the discipline necessary for tackling complex computational problems with confidence and rigour, moving beyond ad-hoc programming, towards principled software development practices.

4 The BOOP Framework for Systematic Problem-Solving

The methodology demonstrated in the previous section forms the foundation of our broader framework: BOOP (Blueprint, Operations, OCaml, Proof). This

structured approach draws inspiration from Knuth’s concept of Literate Programming, which emphasized that programs should be written as literature intended for human understanding rather than mere machine instruction. As Knuth articulated: “Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do.” [20].

Literate programming emphasizes structured design with clear separation of concerns, human-first ordering that prioritizes cognitive comprehension over execution sequence, and meta-language abstraction that builds logical constructs before implementation details. This shift toward human-centric program development aligns with our goals of fostering language-agnostic algorithmic thinking.

4.1 The Four-Phase BOOP Framework

The BOOP framework operationalizes these principles through a mandatory four-phase structure for student submissions:

1. **Blueprint:** Students establish specifications, that is, a functional correctness criterion (preconditions and postconditions). The next iteration of our implementation will support the option for students to add representative test cases as well as state asymptotic complexity bounds.
2. **Operations:** They then design detailed algorithmic steps, expressed in natural language with minimal mathematical notation.
3. **OCaml Code:** Students finally translate their operational steps into an implementation, adhering to constructive programming principles and explicit type annotations. Currently, we support OCaml.
4. **Proof:** Students finally provide a rigorous proof through structural induction or invariant analysis, demonstrating that their implementation satisfies the blueprint specification. This phase closes the loop between specification and verification, embodying the correct-by-construction methodology.

This four-phase BOOP framework prevents the premature leap from problem statement to implementation that characterizes novice programming behaviour. By mandating explicit articulation at each phase, students develop reflective computational thinking practices. The framework’s language-agnostic emphasis prepares students for professional software development’s polyglot reality, where algorithmic insights must transcend programming language syntax.

The BOOP acronym, where the second O stands for ‘OCaml code’ was chosen because the course at our institution is taught in OCaml, and because of the acronym’s appeal and memorability among students. Currently, BOOP supports OCaml but it is a language-agnostic framework for problem-solving, as demonstrated with an example of Java code in Appendix C.

4.2 Enforcement Through Technical Infrastructure

Program Preprocessor Based on experience from Spring 2025, students have been increasingly submitting AI-generated code. This circumvents learning objectives while appearing syntactically correct. The experience motivated us to

develop a preprocessor that promotes conceptual understanding through instructive feedback. Our tool parses OCaml submissions before compilation, identifying counterproductive patterns (such as the use of built-in functions like `List.rev` instead of native implementations, imperative constructs like mutation and variable reassignment that contradict functional programming paradigms, nested functions defined in local scope that obscure algorithmic structure, and functions with anonymous inputs) that prevent clear reasoning about program behaviour.

Furthermore, our system restricts specific syntax and practices that undermine learning goals: it encourages recursive thinking, mandates explicit type reasoning through typed function definitions, promotes first-principles implementation over excessive built-in function usage, and requires pattern matching where it would be more appropriate than if-then-else control flow. These constraints are configurable and can be dynamically adjusted based on assignment objectives, student proficiency levels, and classroom instruction progress. However, while the framework allows for substantial customisation by the instructor, it is not a requirement. Assignments can be deployed with liberal default settings.

Integrated Development Environment On the technical side, the only requirement is installing VS Code, which we found was already the preferred IDE for most students. A custom VS Code extension integrates the BOOP framework with an instructor interface for configuring assignments, specialized syntax highlighting, a completion check to enforce all four phases, and evaluation tools for targeted, conceptual feedback. Snapshots of the IDE and a link to download the bundle are available at Appendix D).

The integrated environment transforms the traditional code-compile-test cycle into a specification-design-implement-verify workflow. We create a learning ecosystem where good practices become the path of least resistance, fostering sustainable computational thinking skills.

5 Observations and Discussion

5.1 Student Learning Outcomes

In the ongoing iteration of the Introduction to Computer Science course, 21 students were asked to submit their solutions to an assignment consisting of functional programming questions using BOOP. The assignment submissions in the previous iteration of the course comprised only of OCaml code and sometimes typeset proofs written in L^AT_EX. We acknowledge our sample size is too small to draw conclusive trends.

Our preliminary evaluation of BOOP reveals significant pedagogical benefits in structured programming assignments. A majority of students demonstrated rigorous algorithmic reasoning and proof skills, with participants successfully constructing sound and complete proofs using the framework’s components. This contrasts sharply with previous cohorts who struggled with proof structure and identification of correctness criteria.

Students reported that BOOP’s scaffolding addressed critical gaps in programming instruction. One participant noted that conventional online tutorials “just start writing code like everyone knows what is to be done,” whereas BOOP provided explicit problem decomposition strategies absent from other learning resources. There were also students who struggled with one part but showed a good grasp of the others. For example, one student could not write code properly but demonstrated good conceptual understanding through blueprint and operational step components. This separation enabled targeted feedback and revealed that coding difficulties often masked sound algorithmic reasoning.

Even incomplete solutions demonstrated adherence to sound programming principles, with students avoiding discouraged practices such as inappropriate built-in function usage. This suggests that BOOP’s preprocessor and emphasis on specification promote disciplined implementation approaches, ultimately encouraging a sustainable thought process over obtaining solutions. Students particularly valued the framework’s ability to clarify the transition between pseudocode and implementation. The structured separation of base and recursive cases addressed a common source of confusion in functional programming. As one student observed, “without BOOP I would forget about correctness and not really think of edge-cases.”

These findings suggest that explicit structural frameworks can significantly improve novice programmers’ formal reasoning abilities while maintaining code quality standards, despite require rigorous elaboration. A student said “BOOP takes four times the time, but I understand the problem much better now.” The framework’s emphasis on correctness-first thinking may help address persistent challenges in computing education related to formal methods integration and long-term computational thinking development.

5.2 Implementation Challenges and Student Misconceptions

Despite these positive outcomes, our evaluation revealed several pedagogical challenges that warrant attention in future implementations.

A significant challenge emerged in students’ approach to the Blueprint component. Rather than developing the correctness required by the problem statement, some students still engaged in code-first approaches, subsequently reverse-engineering the blueprint. Students produced **ensures** clauses that merely restated the implementation logic (sometimes even using the exact if-else clauses written in their functions) rather than abstracting the function’s intended behaviour or establishing correctness criteria independent of their implementation. A student said, “Blueprint just requires you to rewrite the question”, and could not appreciate the skill of developing formal correctness statements.

Our observations show the importance of explicit instruction in formal methods concepts and their practical relevance. It also suggests the need for making lectures tailored to the same approach that assignment submissions entail.

As a part of the evaluation, we also consulted CS educators with decades of experience in teaching introductory courses. All of the consulted educators

recognised value in our framework as it encouraged rigorous articulation of requirements and set the appropriate tone and momentum for a computer science trajectory. Instructors in advanced programming courses offered in our institution (and otherwise [8]) encounter students with inadequate programming practices and weak methods. Early intervention through approaches like BOOP may address these persistent educational challenges and make students well-prepared for future coursework. We continue to explore multi-institutional deployments, comparative analysis with traditional programming pedagogies, longer-term retention studies, and industry preparation assessment.

6 Conclusion and Future Directions

Our work demonstrates that BOOP offers a structured, first-principles approach to computer science education, emphasising reasoning and correctness, over syntax-specific and test-case driven approaches. Several promising research directions emerged from our preliminary evaluation:

1. Integrate BOOP with automated theorem proving and verification environments capable of generating counterexamples and evaluating student-constructed proofs. Tools such as Dafny, Lean, or Why3 are scalable and provide immediate feedback on proof correctness while automatically generating edge cases that challenge student solutions, to establish a comprehensive pipeline supporting automated assessment and systematic feedback. As an easier alternative, be combined with online judges for test-case verification, coming closer to current traditional modes of checking the accuracy of code against a problem statement.
2. To facilitate framework adoption after an initial phase of resistance among students, we plan to tailor lectures to mirror the problem-solving paradigm of BOOP, in an attempt to discourage the notion that BOOP is a final formatting step for assignment submissions only. The perception of BOOP as verbose often stems from students who reverse-engineer correctness criteria, leading to repetition rather than genuine problem analysis. To address this, lectures must emphasise that BOOP components are problem-solving tools, not mere submission requirements. Since students have favored instant feedback, a potential direction is also to use non-suggestive, LLM-driven prompts. We can rate-limit these to discourage frivolous use.

As AI reshapes the landscape of software development, the programmers who will thrive are not those who can compete with machines in code generation, but those who can harness computational thinking to solve problems that machines cannot even formulate. The BOOP structure prepares students for this future by teaching them to think like computer scientists first, and programmers second.

Acknowledgements. We gratefully acknowledge the Mphasis AI & Applied Tech Lab at Ashoka University for supporting this research.

A Three Incorrect Approaches for the Division Function

A.1 Imperative State Tracking:

```

1 let rec minus (n : nat) (m : nat) : nat =
2   match (n, m) with
3   | (Zero, _) -> Zero
4   | (_, Zero) -> n
5   | (Succ n', Succ m') -> minus n' m'
6
7 let rec div1 (a: nat) (b: nat) : nat =
8   match a with
9   | Zero -> Zero
10  | _ -> Succ (div1 (subtract a b) b)

```

This approach reflects the students' tendency to conceptualise division as iterative subtraction, attempting to track a single accumulating value (quotient) rather than the requisite quotient-remainder pair. The three key errors in the function `div1` are:

1. Incorrect quotient if there is a remainder (three divided by two returns two)
2. Division by zero is not handled, and leads to unbounded recursion
3. The remainder is not tracked.

These are caused due to a malformed recursive structure that fails to correctly decompose the problem into its constituent mathematical components.

A.2 Incomplete Edge Case Handling:

```

1 let rec div2 (a: nat) (b: nat) : (nat * nat) =
2   match a with
3   | Zero -> (Zero, Zero)
4   | Succ a' ->
5     let (q, r) = div2 a' b in
6     if (Succ r) < b then (q, Succ r)
7     else (Succ q, Zero)

```

The `div2` function demonstrates a partial understanding of the problem structure, but exhibits critical flaws in specification completeness. Students correctly identify the need for a pair return type but fail to address division by zero, assume the availability of comparison operators without formal definition, and conflate structural recursion on the dividend with the mathematical properties of division.

A.3 Lack of Constructive Foundation:

```

1 let div3 (a: nat) (b: nat) : (nat * nat) = (a / b, a mod b)

```

The function `div3` reveals the difficulty in transitioning from abstract mathematical operations to constructive algorithmic implementations. Students invoke `int` operators for `nat`, demonstrating an incomplete understanding of custom types, type safety, and the nature of the task.

B Proof of Correctness for the Division Function

Theorem 2. *The function `div` in Listing 2 correct; that is, for given natural numbers $a : \text{nat}$ and $b : \text{nat}$:*

1. *If $b = \text{Zero}$, then the function raises a runtime error (division by zero).*
2. *If $b \neq \text{Zero}$, then `div a b` returns a pair (q, r) such that:*
 - (a) $a = (b * q) + r$, and
 - (b) $r < b$.

Proof. We prove the correctness of the function `div` by considering cases:

1. **Case 1:** $b = \text{Zero}$. According to the definition of `div` in Listing 2, line 9:

`match b with Zero -> failwith "div: division by zero"`

the function immediately raises a runtime error.

2. **Case 2:** $b \neq \text{Zero}$. In this case, `div a b` calls `safe_div a b`. We proceed by strong induction on $a : \text{nat}$, that is, we assume the correctness of the function for all strictly smaller inputs $a' < a$ to prove it for a .

Base case: Let $a = \text{Zero}$. Then $\text{Zero} < b$ since $b \neq \text{Zero}$, and thus:

`safe_div Zero b -> (Zero, Zero)`

by pattern matching on `less_than a b`. Hence, `div Zero b` returns $(\text{Zero}, \text{Zero})$. We verify the two postconditions:

- (a) $(b * \text{Zero}) + \text{Zero} = \text{Zero}$, so $a = (b * q) + r$ holds.
- (b) $\text{Zero} < b$, as required.

Inductive Hypothesis: For all $a' : \text{nat}$ such that $a' < a$, assume that `safe_div a' b` returns (q', r') satisfying:

- (a) $a' = (b * q') + r'$, and
- (b) $r' < b$.

Inductive Step: Consider the evaluation of `safe_div a b`.

- (a) **If $a < b$:** Then the function returns (Zero, a) . We check the postconditions:

- (i) $(b * \text{Zero}) + a = a$, so where $q = \text{Zero}$, $r = a$.
- (ii) $a < b$, by assumption.

- (b) **If $a < b = \text{false}$:** Then the function evaluates:

`let (q', r') = safe_div (minus a b) b in (Succ q', r')`

Note that $(a - b)$ is well-defined since $a < b = \text{false}$. Also, we have $(a - b) < a$ for all $b \neq \text{Zero}$, so by the inductive hypothesis applied to $a' = a - b$:

$$a - b = \text{plus } (\text{mult } b \ q') \ r', \quad \text{and} \quad r' < b.$$

Adding b to both sides yields:

$$a = (a - b) + b = ((\text{mult } b \ q') + r') + b.$$

By correctness of addition and multiplication operations:

$$a = (b * (\text{Succ } q')) + r'.$$

Therefore, the returned pair $(\text{Succ } q', r')$ satisfies:

- (i) $a = (b * (\text{Succ } q')) + r'$,
- (ii) $r' < b$.

□

C An Imperative Division Algorithm

This appendix demonstrates how the operational steps developed in section 3.4 can be translated into an imperative paradigm. For clarity and convenience, we will use the arithmetic operators $+$, $*$, and $<$ with the `nat` operations as presented in Listing 1.

The blueprint remains the same:

To write a correct `div`, the function contract is:

1. **Requires:** $a : \text{nat}$, $b : \text{nat}$, and $b \neq \text{Zero}$ (that is b is non-zero).
2. **Ensures:** Returns a pair (q, r) such that:
 - (a) $a = (b * q) + r$
 - (b) `less_than` $r\ b = \text{true}$

The imperative procedure is derived by interpreting the first postcondition, $a = (b * q) + r$, as an invariant. This invariant holds with the initial assignments $q := \text{Zero}$ and $r := a$. Since division can be understood as repeated subtraction, the procedure repeatedly subtracts b from r until the second postcondition, $r < b$, is satisfied. The steps are as follows:

1. Check if $b = \text{Zero}$. In this case return a runtime error.
2. If not, proceed by setting variables q set to `Zero` and r set to a .
3. Until we have $r < b$, repeatedly subtract b from r and each time update $q := \text{Succ } !q$.
4. When $r < b$, return the values for q and r .

The operations can be translated to code. To emphasise on the language-agnostic nature of our approach, we provide the code in OCaml syntax in Listing 3 and Java syntax in Listing 4.

The correctness of the implementation can be presented as a theorem:

Theorem 3. *The function `div` in Listing 2 correct; that is, for given natural numbers $a : \text{nat}$ and $b : \text{nat}$:*

1. If $b = \text{Zero}$, then the function raises a runtime error (division by zero).
2. If $b \neq \text{Zero}$, then `div` $a\ b$ returns a pair (q, r) such that:
 - (a) $a = (b * q) + r$, and
 - (b) $r < b$.

Proof. We analyze the correctness of the implementation using loop invariants.

Case 1: $b = \text{Zero}$. The implementation immediately raises an exception:

```
match b with Zero -> failwith "Division by zero"
```

Thus, the behavior matches the specification.

```

1 let minus (n : nat) (m : nat) : nat =
2   let n_ref = ref n in
3   let m_ref = ref m in
4
5   while !m_ref <> Zero do
6     if !n_ref = Zero then failwith "minus: function requires
7       n >= m"
8     else
9       match !n_ref, !m_ref with
10        | Succ n', Succ m' -> n_ref := n'; m_ref := m'
11        | _ -> assert false
12   done;
13   !n_ref
14
15 let div (a : nat) (b : nat) : (nat * nat) =
16   match b with
17   | Zero -> failwith "Division by zero"
18   | _ ->
19     let q = ref Zero in
20     let r = ref a in
21     while not (less_than !r b) do
22       r := minus !r b;
23       q := Succ !q
24     done;
25     (!q, !r)

```

Listing 3: A translation of the operational steps for imperative division and safe subtraction on natural numbers in OCaml syntax. We assume the availability of a correct `less_than` function for brevity.

Case 2: $b \neq \text{Zero}$. The function proceeds with the following imperative structure:

```

let q = ref Zero in
let r = ref a in
while not (less_than !r b) do
  r := minus !r b;
  q := Succ !q
done;
(!q, !r)

```

We define a loop invariant that must hold at the beginning of each iteration of the loop: $a = (b * !q) + !r$ and $!r : \text{nat}$.

Initialization: Before entering the loop, we have $q = \text{Zero}$ and $r = a$. Then,

$$(b * !q) + !r = (b * \text{Zero}) + a = \text{Zero} + a = a$$

Hence, the invariant holds before the first iteration.

```

1 public class Peano {
2
3     abstract static class Nat {
4         abstract boolean isZero();
5     }
6
7     static final class Zero extends Nat {
8         @Override boolean isZero() { return true; }
9     }
10
11    static final class Succ extends Nat {
12        final Nat prev;
13        Succ(Nat prev) { this.prev = prev; }
14        @Override boolean isZero() { return false; }
15    }
16
17    static Nat minus(Nat n, Nat m) {
18        while (!m.isZero()) {
19            if (n.isZero()) throw new
IllegalArgumentException("minus: n < m");
20            n = ((Succ) n).prev;
21            m = ((Succ) m).prev;
22        }
23        return n;
24    }
25
26    static Nat[] div(Nat a, Nat b) {
27        if (b.isZero()) throw new ArithmeticException("
division by zero");
28        Nat q = new Zero();
29        Nat r = a;
30        while (!lessThan(r, b)) {
31            r = minus(r, b);
32            q = new Succ(q);
33        }
34        return new Nat[] { q, r };
35    }
36 }

```

Listing 4: A translation of the operational steps for division and safe subtraction on natural numbers in Java syntax. We assume the availability of a correct `lessThan` function for brevity.

Maintenance: Assume the invariant holds at the start of an iteration. Then, inside the loop, we update $r := \text{minus } !r \ b$ and $q := \text{Succ } !q$. Let $q_old = !q$ before the iteration and $r_old = !r$ before the iteration. Then after the iteration we have:

$$\begin{aligned} q &:= \text{Succ } q_old \\ r &:= \text{minus } r_old \ b \end{aligned}$$

Using the assumption that $r < b = \text{false}$, the subtraction is valid, and:

$$\begin{aligned} a &= (b * q_old) + r_old = (b * (\text{Succ } q_old)) + (r_old - b) \\ &= (b * q!) + r! \end{aligned}$$

So the invariant continues to hold after each iteration.

Termination: The loop terminates when $!r < b$. At this point, the invariant still holds. Hence the returned pair $(!q, !r)$ satisfies the postconditions. $\square$

D Preview of the IDE

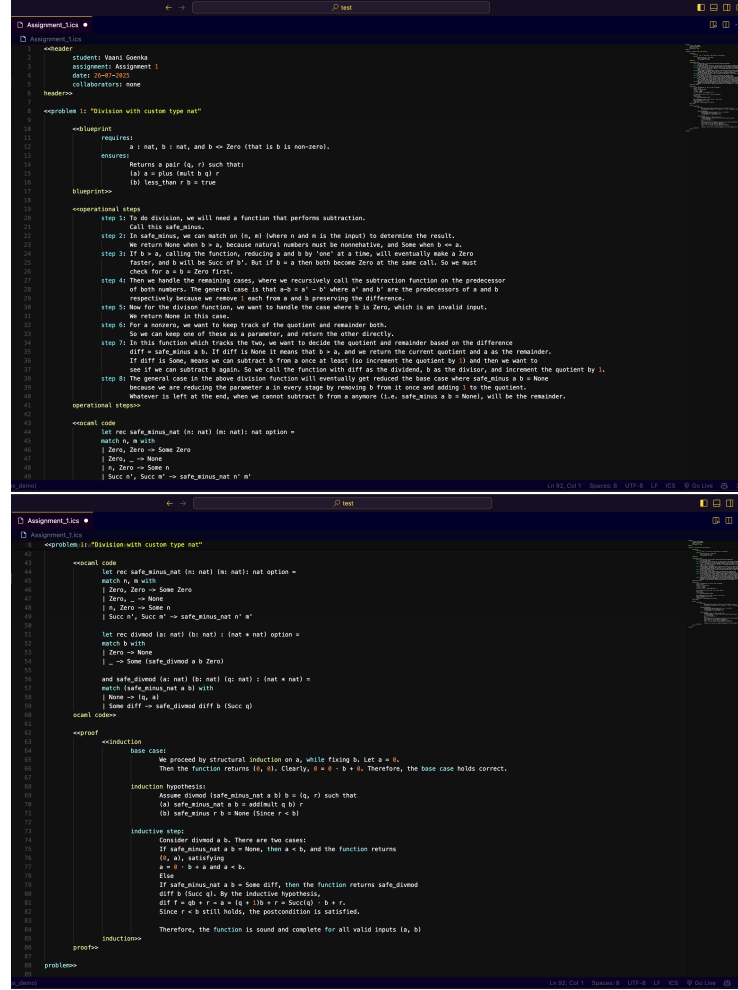


Fig. 1. Screenshots of the VS Code IDE with the ‘BOOP: How to Right Code’ (version 0.5.9) Extension enabled

The complete bundle can be found at zenodo.org/records/16487904. The current version (0.5.9) is a preliminary one, and version 1.0 is under development.

References

1. Amoozadeh, M., Daniels, D., Nam, D., Kumar, A., Chen, S., Hilton, M., Ragan, S.S., Alipour, M.A.: Trust in Generative AI among Students: An exploratory study. In: Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE 2024), vol. 1, pp. 67–73. ACM, New York (2024). <https://doi.org/10.1145/3626252.3630842>
2. Astrachan, O., Mitchener, G., Berry, G., Cox, L.: Design patterns: an essential component of CS curricula. SIGCSE Bulletin. 30(1), 153–160 (1998). <https://doi.org/10.1145/274790.273182>
3. Balse, R., Kumar, V., Prasad, P., Warriem, J.M.: Evaluating the Quality of LLM-Generated Explanations for Logical Errors in CS1 Student Programs. In: Proceedings of the 16th Annual ACM India Compute Conference (COMPUTE '23), pp. 49–54. ACM, New York (2023). <https://doi.org/10.1145/3627217.3627233>
4. Begel, A.: LogoBlocks. (1996)
5. Buschmann, F., Henney, K., Schmidt, D.: Pattern-Oriented Software Architecture: On Patterns And Pattern Language, Volume 5. John Wiley & Sons (2007)
6. Cassano, F., Gouwar, J., Lucchetti, F., Schlesinger, C., Freeman, A., Anderson, C.J., Feldman, M.Q., Greenberg, M., Jangda, A., Guha, A.: Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs. Proceedings of the ACM on Programming Languages. 8(OOPSLA2), 295 (2024). <https://doi.org/10.1145/3689735>
7. Crawford, S., Boese, E.: ActionScript: a gentle introduction to programming. Journal of Computing Sciences in Colleges. 21(3), 156–168 (2006)
8. De Luca, M., et al.: Rookie Mistakes: Measuring Software Quality in Student Projects to Guide Educational Enhancement. arXiv preprint arXiv:2507.12488 (2025)
9. De Marneffe, P.-A.: Le Concept "Holon" et Son Application à la Programmation. Université de Liege (1975)
10. Felleisen, M., Findler, R.B., Flatt, M., Krishnamurthi, S.: How to Design Programs: an Introduction to Programming and Computing. MIT Press, Cambridge (2018)
11. Gamma, E., et al.: Design Patterns: Elements of Reusable Object-Oriented Software. Pearson Deutschland GmbH (1995)
12. Gomes, A., Mendes, A.J.: Problem Solving in Programming. In: Proceedings of the 18th Annual Workshop of the Psychology of Programming Interest Group (PPIG 2007)
13. Gries, D.: The Science of Programming. Springer Science & Business Media (2012)
14. Gries, D., Schneider, F.B.: A Logical Approach to Discrete Math. Springer-Verlag (1993)
15. Haldeman, G.: A synthesized teaching model that promotes both procedural and conceptual knowledge dimensions in CS1. In: Proceedings of the 8th Conference on Computing Education Practice (CEP '24), pp. 9–12. ACM, New York (2024). <https://doi.org/10.1145/3633053.3633064>
16. Hameer, A., Pientka, B.: Teaching the art of functional programming using automated grading (experience report). Proceedings of the ACM on Programming Languages. 3(ICFP), 115 (2019). <https://doi.org/10.1145/3341719>
17. Hoare, C.A.R.: An axiomatic basis for computer programming. Communications of the ACM. 12(10), 576–580 (1969)
18. Hooper, C., et al.: AnnAnn and AnnAnn.Net: Tools for teaching programming. Journal of Computers. 2(5), 9–16 (2007)

19. Hudak, P.: The Haskell School of Expression: Learning Functional Programming Through Multimedia. Cambridge University Press, Cambridge (2000)
20. Knuth, D.E.: Literate Programming. The Computer Journal. 27(2), 97–111 (1984). <https://doi.org/10.1093/comjnl/27.2.97>
21. Knuth, D.E.: Structured Programming with go to Statements. ACM Computing Surveys. 6(4), 261–301 (1974)
22. Knuth, D.E., Levy, S.: The WEB system of structured documentation. Department of Computer Science, Stanford University (1983)
23. Koestler, A.: Beyond atomism and holism—the concept of the holon. Perspectives in Biology and Medicine. 13(2), 131–154 (1970)
24. Kumar, A.N., et al.: Computer Science Curricula 2023. ACM, New York (2024)
25. Lahtinen, E., Ala-Mutka, K., Järvinen, H.-M.: A study of the difficulties of novice programmers. ACM SIGCSE Bulletin. 37(3), 14–18 (2005). <https://doi.org/10.1145/1067445.1067453>
26. Lamport, L.: Who builds a house without drawing blueprints? Communications of the ACM. 58(4), 38–41 (2015)
27. Liu, R., et al.: Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In: Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE 2024), p. 1927. ACM, New York (2024). <https://doi.org/10.1145/3626253.3635427>
28. Loksa, D., et al.: Programming, Problem Solving, and Self-Awareness: Effects of Explicit Guidance. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2016), pp. 1449–1461 (2016)
29. Meyer, B.: Eiffel: The Language. Prentice Hall (1992)
30. Moy, C., Patterson, D.: Teaching Software Specification (Experience Report). Proceedings of the ACM on Programming Languages. 9(ICFP), 264 (2025). <https://doi.org/10.1145/3747533>
31. Noonan, R.E.: Structured programming and formal specification. IEEE Transactions on Software Engineering. SE-1(4), 421–425 (1975). <https://doi.org/10.1109/TSE.1975.6312875>
32. Popovici, M. D.: ChatGPT in the Classroom: Exploring Its Potential and Limitations in a Functional Programming Course. International Journal of Human–Computer Interaction. 40(22), 7743–7754 (2023). <https://doi.org/10.1080/10447318.2023.2269006>
33. Prather, J., et al.: Metacognitive Difficulties Faced by Novice Programmers in Automated Assessment Tools. In: Proceedings of the 13th International Conference on International Computing Education Research (ICER ’18), pp. 41–50. ACM, New York (2018). <https://doi.org/10.1145/3230977.3230981>
34. Raman, A., Kumar, V.: Programming pedagogy and assessment in the era of AI/ML: A position paper. In: Proceedings of the 15th Annual ACM India Compute Conference (COMPUTE ’22), (2022)
35. Rosenblum, D.S.: Formal methods and testing: why the state-of-the-art is not the state-of-the-practice. ACM SIGSOFT Software Engineering Notes. 21(4), 64–66 (1996)
36. Spohrer, J.C., Soloway, E.: Simulating student programmers. In: Proceedings of the 11th International Joint Conference on Artificial intelligence (IJCAI ’89), vol. 1, pp. 525–530 (1989)
37. Weintrop, D.: Block-based programming in computer science education. Communications of the ACM. 62(8), 22–25 (2019). <https://doi.org/10.1145/3341221>
38. Wing, J.M.: Computational thinking. Communications of the ACM. 49(3), 33–35 (2006)